

源程序第 1 页 pyproject.toml

```
===== 文件开始 pyproject.toml =====
0001 [build-system]
0002 requires = ["setuptools>=68"]
0003 build-backend = "setuptools.build_meta"
0004
0005 [project]
0006 name = "llmctg"
0007 version = "1.0.0"
0008 description = "LLM Chinese Text Generator"
0009 readme = "README.md"
0010 requires-python = ">=3.10"
0011 license = {text = "Proprietary"}
0012 dependencies = []
0013
0014 [project.optional-dependencies]
0015 llm = ["torch>=2.1", "transformers>=4.40", "peft>=0.10", "datasets>=2.18"]
0016 dev = ["pytest>=8.0"]
0017
0018 [project.scripts]
0019 llmctg = "llmctg.cli:main"
0020
0021 [tool.setuptools]
0022 package-dir = {"" = "src"}
0023
0024 [tool.setuptools.packages.find]
0025 where = ["src"]
0026
0027 [tool.pytest.ini_options]
0028 pythonpath = ["src"]
0029 testpaths = ["tests"]
0030
===== 文件开始 scripts/run_demo.ps1 =====
0001 $ErrorActionPreference = "Stop"
0002 $projectRoot = Split-Path -Parent $PSScriptRoot
0003 $env:PYTHONPATH = Join-Path $projectRoot "src"
0004 python -m llmctg demo --output (Join-Path $projectRoot "output\demo")
0005
===== 文件开始 scripts/run_demo.sh =====
0001 #!/usr/bin/env bash
0002 set -euo pipefail
0003 PROJECT_ROOT="$(cd "$(dirname "$0")/.." && pwd)"
0004 export PYTHONPATH="$PROJECT_ROOT/src"
0005 python -m llmctg demo --output "$PROJECT_ROOT/output/demo"
0006
===== 文件开始 scripts/run_tests.ps1 =====
0001 $ErrorActionPreference = "Stop"
0002 $projectRoot = Split-Path -Parent $PSScriptRoot
0003 $env:PYTHONPATH = Join-Path $projectRoot "src"
0004 python -m pytest (Join-Path $projectRoot "tests") -q
0005
```

源程序第 2 页 src/llmctg/__init__.py

```
===== 文件开始 src/llmctg/__init__.py =====
0001 """LLM 中文文本生成系统。"""
0002
0003 from .models import ReadingLevel, GenerationRequest, GeneratedArticle, EvaluationResult
0004
0005 __all__ = ["ReadingLevel", "GenerationRequest", "GeneratedArticle", "EvaluationResult"]
0006 __version__ = "1.0.0"
0007
===== 文件开始 src/llmctg/__main__.py =====
0001 from .cli import main
0002
0003 if __name__ == "__main__":
0004     raise SystemExit(main())
0005
===== 文件开始 src/llmctg/cli.py =====
0001 from __future__ import annotations
0002
0003 import argparse
0004 import json
0005 import sys
0006 from pathlib import Path
0007 from typing import Sequence
0008
0009 from . import __version__
0010 from .config import AppConfig
0011 from .dataset import convert_labeled_text
0012 from .io_utils import append_jsonl, write_json
0013 from .logging_utils import configure_logging
0014 from .models import GenerationRequest, ReadingLevel
0015 from .pipeline import LLMCTGApplication
0016 from .reporting import build_summary, write_html_report, write_summary_csv
0017 from .text_utils import read_text
0018 from .training import FineTuneOptions, LoraTrainer
0019
0020
0021 def build_parser() -> argparse.ArgumentParser:
0022     parser = argparse.ArgumentParser(
0023         prog="llmctg",
0024         description="LLM 中文文本生成系统",
0025     )
0026     parser.add_argument("--version", action="version", version=f"%(prog)s {__version__}")
0027     parser.add_argument("--config", help="JSON 配置文件路径")
0028     parser.add_argument("--log-level", choices=["DEBUG", "INFO", "WARNING", "ERROR"])
0029     subparsers = parser.add_subparsers(dest="command", required=True)
0030
0031     generate = subparsers.add_parser("generate", help="生成并评估阅读材料")
0032     _add_generation_arguments(generate)
0033     generate.add_argument("--output", help="输出 JSONL 文件, 省略时打印到终端")
0034
0035     evaluate = subparsers.add_parser("evaluate", help="评估已有文本")
```

源程序第 3 页 src/llmctg/cli.py

```
0036     evaluate_source = evaluate.add_mutually_exclusive_group(required=True)
0037     evaluate_source.add_argument("--text", help="直接传入文本")
0038     evaluate_source.add_argument("--input", help="UTF-8 文本文件")
0039     evaluate.add_argument("--level", default="junior", help="目标等级")
0040     evaluate.add_argument("--output", help="输出 JSON 文件")
0041
0042     batch = subparsers.add_parser("batch", help="执行 JSONL 批量任务")
0043     batch.add_argument("--input", required=True, help="任务 JSONL 文件")
0044     batch.add_argument("--output", required=True, help="结果目录")
0045     batch.add_argument("--no-resume", action="store_true", help="不读取已有结果的任务编号")
0046     _add_backend_arguments(batch)
0047
0048     demo = subparsers.add_parser("demo", help="运行无需模型的完整演示")
0049     demo.add_argument("--output", default="output/demo", help="演示输出目录")
0050
0051     init = subparsers.add_parser("init-config", help="生成默认配置文件")
0052     init.add_argument("--output", default="llmctg.json", help="配置文件路径")
0053
0054     prepare = subparsers.add_parser("prepare-data", help="把制表符标注文本转换为训练 JSONL")
0055     prepare.add_argument("--input", required=True, help="每行格式为 正文 TAB 等级")
0056     prepare.add_argument("--output", required=True, help="训练 JSONL 输出路径")
0057
0058     train = subparsers.add_parser("train", help="使用本地模型执行 LoRA 微调")
0059     train.add_argument("--model-path", required=True)
0060     train.add_argument("--dataset", required=True)
0061     train.add_argument("--output", required=True)
0062     train.add_argument("--epochs", type=float, default=3.0)
0063     train.add_argument("--batch-size", type=int, default=1)
0064     train.add_argument("--max-length", type=int, default=1024)
0065     train.add_argument("--device", default="auto")
0066     return parser
0067
0068
0069 def _add_generation_arguments(parser: argparse.ArgumentParser) -> None:
0070     parser.add_argument("--topic", required=True, help="文章主题")
0071     parser.add_argument("--level", default="junior", help="primary、junior、senior 或中文等级")
0072     parser.add_argument("--instruction", default="", help="额外创作要求")
0073     parser.add_argument("--keywords", default="", help="用逗号分隔的关键词")
0074     parser.add_argument("--count", type=int, default=1, help="生成数量, 1 到 20")
0075     parser.add_argument("--seed", type=int, help="随机种子")
0076     _add_backend_arguments(parser)
0077
0078
0079 def _add_backend_arguments(parser: argparse.ArgumentParser) -> None:
0080     parser.add_argument("--backend", choices=["template", "transformers"], help="生成后端")
0081     parser.add_argument("--model-path", help="本地模型目录")
0082     parser.add_argument("--device", help="auto、cpu、cuda 或 cuda:N")
0083
0084
0085 def main(argv: Sequence[str] | None = None) -> int:
```

源程序第 4 页 src/llmctg/cli.py

```
0086     parser = build_parser()
0087     args = parser.parse_args(argv)
0088     try:
0089         config = AppConfig.load(args.config)
0090         if args.log_level:
0091             config.log_level = args.log_level
0092         _apply_backend_arguments(config, args)
0093         configure_logging(config.log_level)
0094         if args.command == "init-config":
0095             path = config.save(args.output)
0096             print(f"已生成配置文件: {path}")
0097             return 0
0098         if args.command == "prepare-data":
0099             count = convert_labeled_text(args.input, args.output)
0100             print(f"数据转换完成, 共 {count} 条: {args.output}")
0101             return 0
0102         if args.command == "train":
0103             result = LoraTrainer(FineTuneOptions(
0104                 model_path=args.model_path,
0105                 dataset_path=args.dataset,
0106                 output_dir=args.output,
0107                 epochs=args.epochs,
0108                 batch_size=args.batch_size,
0109                 max_length=args.max_length,
0110                 device=args.device,
0111             )).train()
0112             print(json.dumps(result, ensure_ascii=False, indent=2))
0113             return 0
0114         if args.command == "demo":
0115             return _run_demo(config, args)
0116         with LLMCTGApplication(config) as studio:
0117             if args.command == "generate":
0118                 return _run_generate(studio, args)
0119             if args.command == "evaluate":
0120                 return _run_evaluate(studio, args)
0121             if args.command == "batch":
0122                 summary = studio.batch(args.input, args.output, resume=not args.no_resume)
0123                 print(json.dumps(summary.to_dict(), ensure_ascii=False, indent=2))
0124                 return 0 if summary.processing_errors == 0 else 2
0125         parser.error("未知命令")
0126     except (ValueError, FileNotFoundError, RuntimeError) as error:
0127         print(f"错误: {error}", file=sys.stderr)
0128         return 2
0129     return 0
0130
0131
0132 def _apply_backend_arguments(config: AppConfig, args: argparse.Namespace) -> None:
0133     for name in ("backend", "model_path", "device"):
0134         value = getattr(args, name, None)
0135         if value:
```

源程序第 5 页 src/llmctg/cli.py

```
0136         setattr(config.generation, name, value)
0137
0138
0139 def _run_generate(studio: LLMCTGApplication, args: argparse.Namespace) -> int:
0140     keywords = [item.strip() for item in args.keywords.replace(", ", ",").split(",") if item.strip()]
0141     request = GenerationRequest(
0142         topic=args.topic,
0143         level=ReadingLevel.parse(args.level),
0144         instruction=args.instruction,
0145         keywords=keywords,
0146         count=args.count,
0147         seed=args.seed,
0148     )
0149     articles = studio.generate(request)
0150     for article in articles:
0151         if args.output:
0152             append_jsonl(args.output, article.to_dict())
0153         else:
0154             print(json.dumps(article.to_dict(), ensure_ascii=False, indent=2))
0155     return 0
0156
0157
0158 def _run_evaluate(studio: LLMCTGApplication, args: argparse.Namespace) -> int:
0159     text = args.text if args.text is not None else read_text(args.input)
0160     result = studio.evaluate(text, ReadingLevel.parse(args.level))
0161     payload = result.to_dict()
0162     if args.output:
0163         write_json(args.output, payload)
0164         print(f"评估结果已保存: {args.output}")
0165     else:
0166         print(json.dumps(payload, ensure_ascii=False, indent=2))
0167     return 0 if result.passed else 1
0168
0169
0170 def _run_demo(config: AppConfig, args: argparse.Namespace) -> int:
0171     config.generation.backend = "template"
0172     output = Path(args.output)
0173     output.mkdir(parents=True, exist_ok=True)
0174     articles = []
0175     with LLMCTGApplication(config) as studio:
0176         for index, (topic, level) in enumerate([
0177             ("四季的变化", ReadingLevel.PRIMARY),
0178             ("城市里的河流", ReadingLevel.JUNIOR),
0179             ("人工智能与学习", ReadingLevel.SENIOR),
0180         ]):
0181             request = GenerationRequest(topic=topic, level=level, count=1, seed=2026 + index)
0182             generated = studio.generate(request)
0183             articles.extend(generated)
0184             append_jsonl(output / "articles.jsonl", generated[0].to_dict())
0185     summary = build_summary(articles)
```

源程序第 6 页 src/llmctg/cli.py

```
0186     write_json(output / "summary.json", summary.to_dict())
0187     write_summary_csv(output / "summary.csv", articles)
0188     write_html_report(output / "report.html", articles, summary)
0189     print(f"演示完成, 结果目录: {output.resolve()}")
0190     return 0
0191
0192
0193 if __name__ == "__main__":
0194     raise SystemExit(main())
0195
0196 ===== 文件开始 src/llmctg/config.py =====
0001 from __future__ import annotations
0002
0003 import json
0004 from dataclasses import asdict, dataclass, field
0005 from pathlib import Path
0006 from typing import Any
0007
0008
0009 @dataclass(slots=True)
0010 class GenerationConfig:
0011     backend: str = "template"
0012     model_path: str = ""
0013     device: str = "auto"
0014     temperature: float = 0.9
0015     top_p: float = 0.9
0016     top_k: int = 50
0017     repetition_penalty: float = 1.1
0018     max_new_tokens_primary: int = 380
0019     max_new_tokens_junior: int = 800
0020     max_new_tokens_senior: int = 1200
0021
0022
0023 @dataclass(slots=True)
0024 class EvaluationConfig:
0025     pass_score: float = 60.0
0026     excellent_score: float = 85.0
0027     good_score: float = 70.0
0028     primary_sentence_target: float = 16.0
0029     junior_sentence_target: float = 24.0
0030     senior_sentence_target: float = 34.0
0031     repetition_warning: float = 0.18
0032     english_warning: float = 0.15
0033
0034
0035 @dataclass(slots=True)
0036 class AppConfig:
0037     generation: GenerationConfig = field(default_factory=GenerationConfig)
0038     evaluation: EvaluationConfig = field(default_factory=EvaluationConfig)
0039     log_level: str = "INFO"
```

源程序第 7 页 src/llmctg/config.py

```
0040     output_encoding: str = "utf-8"
0041
0042     @classmethod
0043     def load(cls, path: str | Path | None = None) -> "AppConfig":
0044         if not path:
0045             return cls()
0046         config_path = Path(path)
0047         if not config_path.exists():
0048             raise FileNotFoundError(f"配置文件不存在: {config_path}")
0049         with config_path.open("r", encoding="utf-8") as stream:
0050             raw = json.load(stream)
0051         return cls(
0052             generation=GenerationConfig(**raw.get("generation", {})),
0053             evaluation=EvaluationConfig(**raw.get("evaluation", {})),
0054             log_level=raw.get("log_level", "INFO"),
0055             output_encoding=raw.get("output_encoding", "utf-8"),
0056         )
0057
0058     def save(self, path: str | Path) -> Path:
0059         target = Path(path)
0060         target.parent.mkdir(parents=True, exist_ok=True)
0061         with target.open("w", encoding="utf-8") as stream:
0062             json.dump(asdict(self), stream, ensure_ascii=False, indent=2)
0063         return target
0064
0065     def merge(self, values: dict[str, Any]) -> "AppConfig":
0066         for section, section_values in values.items():
0067             target = getattr(self, section, None)
0068             if target is None or not isinstance(section_values, dict):
0069                 continue
0070             for name, value in section_values.items():
0071                 if hasattr(target, name) and value is not None:
0072                     setattr(target, name, value)
0073         return self
0074
0075 ===== 文件开始 src/llmctg/dataset.py =====
0001 from __future__ import annotations
0002
0003 import json
0004 from dataclasses import dataclass
0005 from pathlib import Path
0006 from typing import Iterator
0007
0008 from .models import ReadingLevel
0009
0010
0011 @dataclass(slots=True)
0012 class TrainingSample:
0013     instruction: str
0014     input: str
```

源程序第 8 页 src/llmctg/dataset.py

```
0015     output: str
0016     level: ReadingLevel
0017
0018     def to_dict(self) -> dict[str, str]:
0019         return {
0020             "instruction": self.instruction,
0021             "input": self.input,
0022             "output": self.output,
0023             "level": self.level.value,
0024         }
0025
0026
0027 def parse_labeled_text(path: str | Path) -> Iterator[TrainingSample]:
0028     source = Path(path)
0029     with source.open("r", encoding="utf-8-sig") as stream:
0030         for line_number, line in enumerate(stream, 1):
0031             stripped = line.strip()
0032             if not stripped:
0033                 continue
0034             try:
0035                 text, raw_level = stripped.rsplit("\t", 1)
0036             except ValueError as error:
0037                 raise ValueError(f"{source} 第 {line_number} 行缺少制表符和等级标签") from error
0038             level = ReadingLevel.parse(raw_level)
0039             text = text.strip()
0040             if not text:
0041                 raise ValueError(f"{source} 第 {line_number} 行正文为空")
0042             yield TrainingSample(
0043                 instruction=f"请生成一段适合{level.chinese_name}学生阅读的中文内容。",
0044                 input="",
0045                 output=text,
0046                 level=level,
0047             )
0048
0049
0050 def convert_labeled_text(input_path: str | Path, output_path: str | Path) -> int:
0051     samples = list(parse_labeled_text(input_path))
0052     target = Path(output_path)
0053     target.parent.mkdir(parents=True, exist_ok=True)
0054     with target.open("w", encoding="utf-8", newline="\n") as stream:
0055         for sample in samples:
0056             stream.write(json.dumps(sample.to_dict(), ensure_ascii=False) + "\n")
0057     return len(samples)
0058
0059
0060 ===== 文件开始 src/llmctg/evaluator.py =====
0001 from __future__ import annotations
0002
0003 import math
0004 import re
0005 from collections import Counter
```

源程序第 9 页 src/llmctg/evaluator.py

```
0006
0007 from .config import EvaluationConfig
0008 from .models import EvaluationResult, QualityMetrics, ReadingLevel, TextStatistics
0009 from .text_utils import chinese_characters, normalize_text, repetition_rate, split_paragraphs, split_sentences
0010
0011 COMMON_CHARACTERS = set(
0012     "的一是在不了有和人这中大为上国我以要他时来用们生到作地于出就分对成会可主发年动同工也能下过子说产种面而方后多定行学法所民得经十三之进着等部度家电力里
0013     如水化高自二理起小物现实加量都两体制机当使点从业本去把性好应开它合还因由某些然前外天政四日那社义事平形相全表间样与关各重新线内数正心反你明看原又么利比或但质气第向
0014     道命此变条只没结解问意建月公无系军很情者最立代想已通并提直题党程展五果料象员革位入常文总次品式活设及管特件长求老头基资边流路级少图山统接知较将组见计别她手角期根论
0015     运农指几九区强放决西被干做必战先回则任取据处理世车更斗百给名真论条形"
0016 )
0017 PUNCTUATION = set(",。!?!?;:、.!?;:‘’""()《》〈〉-.-\n")
0018
0019 class ReadabilityEvaluator:
0020     def __init__(self, config: EvaluationConfig | None = None):
0021         self.config = config or EvaluationConfig()
0022
0023     def evaluate(self, text: str, target_level: ReadingLevel | str | int) -> EvaluationResult:
0024         target = ReadingLevel.parse(target_level)
0025         normalized = normalize_text(text)
0026         statistics = self._statistics(normalized)
0027         predicted = self._predict_level(statistics, normalized)
0028         metrics = self._metrics(normalized, target, predicted, statistics)
0029         warnings, suggestions = self._advice(normalized, target, statistics, metrics)
0030         grade = self._grade(metrics.overall_score)
0031         passed = metrics.overall_score >= self.config.pass_score and len(normalized) > 0
0032         return EvaluationResult(
0033             target_level=target,
0034             predicted_level=predicted,
0035             quality_grade=grade,
0036             passed=passed,
0037             statistics=statistics,
0038             metrics=metrics,
0039             warnings=warnings,
0040             suggestions=suggestions,
0041         )
0042
0043     def _statistics(self, text: str) -> TextStatistics:
0044         chars = chinese_characters(text)
0045         sentences = split_sentences(text)
0046         paragraphs = split_paragraphs(text)
0047         lengths = [len(chinese_characters(item)) for item in sentences]
0048         unique_ratio = len(set(chars)) / len(chars) if chars else 0.0
0049         punctuation_count = sum(1 for char in text if char in PUNCTUATION)
0050         return TextStatistics(
0051             characters=len(text.replace("\n", "")),
0052             chinese_characters=len(chars),
0053             sentences=len(sentences),
0054             paragraphs=len(paragraphs),
0055             average_sentence_length=sum(lengths) / len(lengths) if lengths else 0.0,
0056             max_sentence_length=max(lengths, default=0),
0057             unique_character_ratio=round(unique_ratio, 4),
```

源程序第 10 页 src/llmctg/evaluator.py

```
0056         punctuation_ratio=round(punctuation_count / len(text), 4) if text else 0.0,
0057     )
0058
0059     def _predict_level(self, stats: TextStatistics, text: str) -> ReadingLevel:
0060         rare_ratio = self._uncommon_ratio(text)
0061         complexity = stats.average_sentence_length + rare_ratio * 55 + max(0, stats.max_sentence_length - 30) * 0.12
0062         if complexity < 20:
0063             return ReadingLevel.PRIMARY
0064         if complexity < 31:
0065             return ReadingLevel.JUNIOR
0066         return ReadingLevel.SENIOR
0067
0068     def _metrics(
0069         self,
0070         text: str,
0071         target: ReadingLevel,
0072         predicted: ReadingLevel,
0073         stats: TextStatistics,
0074     ) -> QualityMetrics:
0075         repeat = repetition_rate(text, size=4)
0076         english_count = sum(1 for char in text if "a" <= char.lower() <= "z")
0077         english = english_count / max(1, len(text))
0078         uncommon = self._uncommon_ratio(text)
0079         structure = self._structure_score(text, stats)
0080         length = self._length_score(target, stats.chinese_characters)
0081         distance = abs(predicted.numeric_label - target.numeric_label)
0082         level_score = max(0.0, 100.0 - distance * 38.0)
0083         fluency_score = max(0.0, 100.0 - repeat * 260.0 - english * 100.0 - uncommon * 65.0)
0084         overall = structure * 0.20 + length * 0.20 + level_score * 0.30 + fluency_score * 0.30
0085         return QualityMetrics(
0086             repetition_rate=round(repeat, 4),
0087             english_ratio=round(english, 4),
0088             uncommon_character_ratio=round(uncommon, 4),
0089             structure_score=round(structure, 2),
0090             length_score=round(length, 2),
0091             level_score=round(level_score, 2),
0092             overall_score=round(overall, 2),
0093         )
0094
0095     @staticmethod
0096     def _uncommon_ratio(text: str) -> float:
0097         chars = chinese_characters(text)
0098         if not chars:
0099             return 0.0
0100         return sum(1 for char in chars if char not in COMMON_CHARACTERS) / len(chars)
0101
0102     @staticmethod
0103     def _structure_score(text: str, stats: TextStatistics) -> float:
0104         if not text:
0105             return 0.0
```

```

0106     score = 40.0
0107     if stats.sentences >= 3:
0108         score += 25
0109     if stats.paragraphs >= 2:
0110         score += 20
0111     if text[-1] in ". ! ? !?...\"'\"'\"' ] ) )":
0112         score += 10
0113     if re.search(r"[, , ; : ]", text):
0114         score += 5
0115     return min(100.0, score)
0116
0117 @staticmethod
0118 def _length_score(level: ReadingLevel, length: int) -> float:
0119     targets = {
0120         ReadingLevel.PRIMARY: (120, 500),
0121         ReadingLevel.JUNIOR: (250, 1000),
0122         ReadingLevel.SENIOR: (400, 1800),
0123     }
0124     minimum, maximum = targets[level]
0125     if minimum <= length <= maximum:
0126         return 100.0
0127     if length < minimum:
0128         return max(0.0, length / minimum * 100.0)
0129     overflow = (length - maximum) / maximum
0130     return max(40.0, 100.0 - overflow * 70.0)
0131
0132 def _grade(self, score: float) -> str:
0133     if score >= self.config.excellent_score:
0134         return "优秀"
0135     if score >= self.config.good_score:
0136         return "良好"
0137     if score >= self.config.pass_score:
0138         return "合格"
0139     return "待改进"
0140
0141 def _advice(
0142     self,
0143     text: str,
0144     target: ReadingLevel,
0145     stats: TextStatistics,
0146     metrics: QualityMetrics,
0147 ) -> tuple[list[str], list[str]]:
0148     warnings: list[str] = []
0149     suggestions: list[str] = []
0150     if not text:
0151         return ["文本为空"], ["请输入需要评估的完整中文文本"]
0152     if metrics.repetition_rate > self.config.repetition_warning:
0153         warnings.append("连续片段重复比例偏高")
0154         suggestions.append("删除机械重复句，使用指代或改写衔接相近内容")
0155     if metrics.english_ratio > self.config.english_warning:

```

源程序第 12 页 src/llmctg/evaluator.py

```
0156         warnings.append("英文字符比例偏高")
0157         suggestions.append("确认英文术语是否必要，并在首次出现时提供中文解释")
0158     sentence_targets = {
0159         ReadingLevel.PRIMARY: self.config.primary_sentence_target,
0160         ReadingLevel.JUNIOR: self.config.junior_sentence_target,
0161         ReadingLevel.SENIOR: self.config.senior_sentence_target,
0162     }
0163     if stats.average_sentence_length > sentence_targets[target] * 1.35:
0164         warnings.append("平均句长高于目标等级建议值")
0165         suggestions.append("拆分长句，减少连续修饰成分和多层转折")
0166     if stats.paragraphs < 2 and stats.chinese_characters >= 100:
0167         warnings.append("长文本缺少段落划分")
0168         suggestions.append("按引入、说明和总结划分自然段")
0169     if metrics.length_score < 70:
0170         warnings.append("文本长度与目标等级不匹配")
0171         suggestions.append("围绕主题补充事实、例子或总结，避免无关扩写")
0172     if metrics.level_score < 80:
0173         warnings.append("预测难度与目标阅读等级存在差异")
0174         direction = "降低" if stats.average_sentence_length > sentence_targets[target] else "提高"
0175         suggestions.append(f"通过词语解释、句式调整和信息密度控制来{direction}阅读难度")
0176     return warnings, list(dict.fromkeys(suggestions))
0177
===== 文件开始 src/llmctg/generators/__init__.py =====
0001 from .base import TextGenerator
0002 from .factory import create_generator
0003 from .template import TemplateGenerator
0004
0005 __all__ = ["TextGenerator", "TemplateGenerator", "create_generator"]
0006
===== 文件开始 src/llmctg/generators/base.py =====
0001 from __future__ import annotations
0002
0003 from abc import ABC, abstractmethod
0004
0005 from ..models import GenerationRequest
0006
0007
0008 class TextGenerator(ABC):
0009     name = "base"
0010
0011     @abstractmethod
0012     def generate(self, request: GenerationRequest) -> list[tuple[str, str]]:
0013         """返回标题、正文组成的列表。"""
0014
0015     def close(self) -> None:
0016         """释放可选的模型资源。"""
0017
===== 文件开始 src/llmctg/generators/factory.py =====
0001 from __future__ import annotations
0002
```

源程序第 13 页 src/llmctg/generators/factory.py

```
0003 from ..config import GenerationConfig
0004 from .base import TextGenerator
0005 from .template import TemplateGenerator
0006
0007
0008 def create_generator(config: GenerationConfig) -> TextGenerator:
0009     backend = config.backend.strip().lower()
0010     if backend in {"template", "demo", "offline"}:
0011         return TemplateGenerator()
0012     if backend in {"transformers", "huggingface", "local-model"}:
0013         from .transformers_backend import TransformersGenerator
0014         return TransformersGenerator(config)
0015     raise ValueError(f"不支持的生成后端: {config.backend}")
0016
0017 ===== 文件开始 src/llmctg/generators/template.py =====
0001 from __future__ import annotations
0002
0003 import random
0004
0005 from ..models import GenerationRequest, ReadingLevel
0006 from ..text_utils import ensure_terminal_punctuation
0007 from .base import TextGenerator
0008
0009
0010 class TemplateGenerator(TextGenerator):
0011     """用于离线演示和安装验收的确定性生成器。"""
0012
0013     name = "template"
0014
0015     OPENINGS = {
0016         ReadingLevel.PRIMARY: [
0017             "今天，我们一起认识{topic}。",
0018             "你注意过{topic}吗？它就在我们的生活里。",
0019             "关于{topic}，有许多有趣的小秘密。",
0020         ],
0021         ReadingLevel.JUNIOR: [
0022             "当我们讨论{topic}时，首先要理解它与日常生活的联系。",
0023             "{topic}看似熟悉，其中却包含值得追问的规律。",
0024             "从一个常见现象出发，我们可以逐步认识{topic}。",
0025         ],
0026         ReadingLevel.SENIOR: [
0027             "理解{topic}，需要同时观察事实、机制及其产生的社会影响。",
0028             "{topic}并非孤立现象，它由多种条件共同塑造。",
0029             "围绕{topic}的讨论，往往涉及科学判断与价值选择的结合。",
0030         ],
0031     }
0032     OBSERVATIONS = {
0033         ReadingLevel.PRIMARY: [
0034             "我们可以先看一看，再想一想它为什么会这样。",
0035             "仔细观察，会发现变化不是一下子发生的。",
```

源程序第 14 页 src/llmctg/generators/template.py

```
0036         "把看到的事情记下来，就能发现新的线索。",
0037     ],
0038     ReadingLevel.JUNIOR: [
0039         "观察现象只是第一步，还要比较条件、记录变化并寻找原因。",
0040         "同一现象在不同环境中可能出现差异，因此结论需要证据支持。",
0041         "提出问题、收集资料和验证猜想，可以帮助我们避免凭感觉判断。",
0042     ],
0043     ReadingLevel.SENIOR: [
0044         "分析这一主题时，应区分相关关系与因果关系，并说明证据的适用范围。",
0045         "可靠的结论来自可重复的观察、清晰的概念和对反例的认真检验。",
0046         "不同尺度上的机制可能彼此影响，单一解释通常不足以覆盖全部情形。",
0047     ],
0048 }
0049 ACTIONS = {
0050     ReadingLevel.PRIMARY: [
0051         "我们可以从身边的小事做起，把问题写进观察本。",
0052         "遇到不懂的地方，可以查资料，也可以请教老师和同学。",
0053         "只要愿意动手试一试，知识就会变得更清楚。",
0054     ],
0055     ReadingLevel.JUNIOR: [
0056         "实践时可以设置一个明确目标，并用相同标准记录每次结果。",
0057         "阅读多种资料、核对来源，再用自己的语言概括，是有效的学习方法。",
0058         "如果结果与预期不同，不妨检查步骤并修正原来的假设。",
0059     ],
0060     ReadingLevel.SENIOR: [
0061         "进一步研究可以建立指标体系，比较不同方案的收益、成本与不确定性。",
0062         "面对相互冲突的材料，应检查样本、方法和论证链条，而非只看结论。",
0063         "将理论解释转化为可检验的问题，有助于形成审慎而开放的判断。",
0064     ],
0065 }
0066 CLOSINGS = {
0067     ReadingLevel.PRIMARY: [
0068         "让我们带着好奇心，继续发现{topic}的故事吧！",
0069         "下一次见到它时，你也许会有新的发现。",
0070     ],
0071     ReadingLevel.JUNIOR: [
0072         "由此可见，认识{topic}既需要知识，也需要耐心和实践。",
0073         "持续观察和主动求证，会让我们对{topic}形成更完整的认识。",
0074     ],
0075     ReadingLevel.SENIOR: [
0076         "因此，对{topic}的理解应随着证据更新，并在具体情境中接受检验。",
0077         "只有把事实、逻辑与责任结合起来，我们才能更稳妥地回应相关问题。",
0078     ],
0079 }
0080
0081 def generate(self, request: GenerationRequest) -> list[tuple[str, str]]:
0082     request.validate()
0083     randomizer = random.Random(request.seed)
0084     results: list[tuple[str, str]] = []
0085     for index in range(request.count):
```

源程序第 15 页 src/llmctg/generators/template.py

```
0086         title = self._title(request, index)
0087         paragraphs = self._paragraphs(request, randomizer)
0088         results.append((title, ensure_terminal_punctuation("\n".join(paragraphs))))
0089     return results
0090
0091     def _title(self, request: GenerationRequest, index: int) -> str:
0092         suffix = "" if request.count == 1 else f" ({index + 1}) "
0093         names = {
0094             ReadingLevel.PRIMARY: f"认识{request.topic}",
0095             ReadingLevel.JUNIOR: f"探索{request.topic}",
0096             ReadingLevel.SENIOR: f"理解{request.topic}的多重视角",
0097         }
0098         return names[request.level] + suffix
0099
0100     def _paragraphs(self, request: GenerationRequest, randomizer: random.Random) -> list[str]:
0101         context = {"topic": request.topic}
0102         opening = randomizer.choice(self.OPENINGS[request.level]).format(**context)
0103         observation = randomizer.choice(self.OBSERVATIONS[request.level]).format(**context)
0104         action = randomizer.choice(self.ACTIONS[request.level]).format(**context)
0105         closing = randomizer.choice(self.CLOSINGS[request.level]).format(**context)
0106         keyword_sentence = self._keyword_sentence(request)
0107         instruction_sentence = self._instruction_sentence(request)
0108         if request.level == ReadingLevel.PRIMARY:
0109             return [opening + keyword_sentence, observation, action, closing]
0110         if request.level == ReadingLevel.JUNIOR:
0111             return [opening + keyword_sentence, observation + instruction_sentence, action, closing]
0112         extension = (
0113             f"以{request.topic}为例，个体经验能够提出问题，但仍需通过系统资料来检验。"
0114             "当条件改变时，原有解释也可能需要调整，这正是理性探究的价值所在。"
0115         )
0116         return [opening + keyword_sentence, observation + instruction_sentence, extension, action, closing]
0117
0118     @staticmethod
0119     def _keyword_sentence(request: GenerationRequest) -> str:
0120         if not request.keywords:
0121             return ""
0122         joined = "、".join(request.keywords[:6])
0123         return f"阅读时可以留意这些关键词：{joined}。"
0124
0125     @staticmethod
0126     def _instruction_sentence(request: GenerationRequest) -> str:
0127         if not request.instruction.strip():
0128             return ""
0129         return f"本次阅读任务是：{request.instruction.strip()}。"
0130
0131 ===== 文件开始 src/llmctg/generators/transformers_backend.py =====
0001 from __future__ import annotations
0002
0003 from typing import Any
0004
```

源程序第 16 页 src/llmctg/generators/transformers_backend.py

```
0005 from ..config import GenerationConfig
0006 from ..models import GenerationRequest, ReadingLevel
0007 from ..text_utils import ensure_terminal_punctuation
0008 from .base import TextGenerator
0009
0010
0011 class TransformersGenerator(TextGenerator):
0012     """惰性加载本地 Hugging Face 模型的可选生成后端。"""
0013
0014     name = "transformers"
0015
0016     def __init__(self, config: GenerationConfig):
0017         if not config.model_path:
0018             raise ValueError("使用 transformers 后端时必须指定 model_path")
0019         self.config = config
0020         self._torch: Any = None
0021         self._tokenizer: Any = None
0022         self._model: Any = None
0023
0024     def _load(self) -> None:
0025         if self._model is not None:
0026             return
0027         try:
0028             import torch
0029             from transformers import AutoModelForCausalLM, AutoTokenizer
0030         except ImportError as error:
0031             raise RuntimeError("缺少模型依赖, 请执行 pip install -e .[llm]") from error
0032         self._torch = torch
0033         self._tokenizer = AutoTokenizer.from_pretrained(
0034             self.config.model_path, trust_remote_code=False, use_fast=True
0035         )
0036         device = self.config.device
0037         if device == "auto":
0038             device = "cuda" if torch.cuda.is_available() else "cpu"
0039         dtype = torch.float16 if str(device).startswith("cuda") else torch.float32
0040         self._model = AutoModelForCausalLM.from_pretrained(
0041             self.config.model_path, trust_remote_code=False, torch_dtype=dtype
0042         ).to(device)
0043         self._model.eval()
0044         if self._tokenizer.pad_token_id is None:
0045             self._tokenizer.pad_token = self._tokenizer.eos_token
0046
0047     def generate(self, request: GenerationRequest) -> list[tuple[str, str]]:
0048         request.validate()
0049         self._load()
0050         return [self._generate_one(request, index) for index in range(request.count)]
0051
0052     def _generate_one(self, request: GenerationRequest, index: int) -> tuple[str, str]:
0053         prompt = self._build_prompt(request)
0054         device = next(self._model.parameters()).device
```

源程序第 17 页 src/llmctg/generators/transformers_backend.py

```
0055         encoded = self._tokenizer(prompt, return_tensors="pt", truncation=True, max_length=2048)
0056         encoded = {key: value.to(device) for key, value in encoded.items()}
0057         maximums = {
0058             ReadingLevel.PRIMARY: self.config.max_new_tokens_primary,
0059             ReadingLevel.JUNIOR: self.config.max_new_tokens_junior,
0060             ReadingLevel.SENIOR: self.config.max_new_tokens_senior,
0061         }
0062         generator = None
0063         if request.seed is not None:
0064             generator = self._torch.Generator(device=device).manual_seed(request.seed + index)
0065         with self._torch.inference_mode():
0066             output = self._model.generate(
0067                 **encoded,
0068                 do_sample=True,
0069                 max_new_tokens=maximums[request.level],
0070                 temperature=self.config.temperature,
0071                 top_p=self.config.top_p,
0072                 top_k=self.config.top_k,
0073                 repetition_penalty=self.config.repetition_penalty,
0074                 pad_token_id=self._tokenizer.pad_token_id,
0075                 eos_token_id=self._tokenizer.eos_token_id,
0076                 generator=generator,
0077             )
0078         new_tokens = output[0, encoded["input_ids"].shape[-1]:]
0079         content = self._tokenizer.decode(new_tokens, skip_special_tokens=True).strip()
0080         title = f"{request.topic}分级阅读材料"
0081         return title, ensure_terminal_punctuation(content)
0082
0083     @staticmethod
0084     def _build_prompt(request: GenerationRequest) -> str:
0085         keywords = " ".join(request.keywords) if request.keywords else "无指定关键词"
0086         extra = request.instruction.strip() or "内容准确、结构完整、表达自然"
0087         return (
0088             f"请围绕“{request.topic}”创作一篇适合{request.level.chinese_name}学生阅读的中文文章。\\n"
0089             f"关键词: {keywords}\\n 要求: {extra}\\n 只输出文章正文, 不要解释创作过程。\\n 正文: "
0090         )
0091
0092     def close(self) -> None:
0093         self._model = None
0094         self._tokenizer = None
0095         if self._torch is not None and self._torch.cuda.is_available():
0096             self._torch.cuda.empty_cache()
0097
0098     ===== 文件开始 src/llmctg/io_utils.py =====
0001     from __future__ import annotations
0002
0003     import csv
0004     import json
0005     from pathlib import Path
0006     from typing import Any, Iterable, Iterator
```

```

0007
0008
0009 class DataFormatError(ValueError):
0010     """输入记录无法解析时抛出。"""
0011
0012
0013 def read_jsonl(path: str | Path) -> Iterator[dict[str, Any]]:
0014     source = Path(path)
0015     if not source.exists():
0016         raise FileNotFoundError(f"输入文件不存在: {source}")
0017     with source.open("r", encoding="utf-8-sig") as stream:
0018         for line_number, line in enumerate(stream, 1):
0019             stripped = line.strip()
0020             if not stripped:
0021                 continue
0022             try:
0023                 value = json.loads(stripped)
0024             except json.JSONDecodeError as error:
0025                 raise DataFormatError(f"{source} 第 {line_number} 行不是有效 JSON: {error.msg}") from error
0026             if not isinstance(value, dict):
0027                 raise DataFormatError(f"{source} 第 {line_number} 行必须是 JSON 对象")
0028             value["_line_number"] = line_number
0029             yield value
0030
0031
0032 def append_jsonl(path: str | Path, value: dict[str, Any]) -> Path:
0033     target = Path(path)
0034     target.parent.mkdir(parents=True, exist_ok=True)
0035     with target.open("a", encoding="utf-8", newline="\n") as stream:
0036         stream.write(json.dumps(value, ensure_ascii=False, separators=(",", ":")) + "\n")
0037     return target
0038
0039
0040 def write_json(path: str | Path, value: Any) -> Path:
0041     target = Path(path)
0042     target.parent.mkdir(parents=True, exist_ok=True)
0043     with target.open("w", encoding="utf-8", newline="\n") as stream:
0044         json.dump(value, stream, ensure_ascii=False, indent=2)
0045         stream.write("\n")
0046     return target
0047
0048
0049 def write_csv(path: str | Path, rows: Iterable[dict[str, Any]], fieldnames: list[str]) -> Path:
0050     target = Path(path)
0051     target.parent.mkdir(parents=True, exist_ok=True)
0052     with target.open("w", encoding="utf-8-sig", newline="") as stream:
0053         writer = csv.DictWriter(stream, fieldnames=fieldnames, extrasaction="ignore")
0054         writer.writeheader()
0055         writer.writerows(rows)
0056     return target

```

源程序第 19 页 src/llmctg/io_utils.py

```
0057
0058
0059 def existing_ids(path: str | Path, key: str = "request_id") -> set[str]:
0060     target = Path(path)
0061     if not target.exists():
0062         return set()
0063     found: set[str] = set()
0064     try:
0065         for record in read_jsonl(target):
0066             value = record.get(key)
0067             if value:
0068                 found.add(str(value))
0069     except DataFormatError:
0070         return found
0071     return found
0072
==== 文件开始 src/llmctg/logging_utils.py ====
0001 from __future__ import annotations
0002
0003 import logging
0004 from pathlib import Path
0005
0006
0007 def configure_logging(level: str = "INFO", log_file: str | Path | None = None) -> None:
0008     handlers: list[logging.Handler] = [logging.StreamHandler()]
0009     if log_file:
0010         target = Path(log_file)
0011         target.parent.mkdir(parents=True, exist_ok=True)
0012         handlers.append(logging.FileHandler(target, encoding="utf-8"))
0013     logging.basicConfig(
0014         level=getattr(logging, level.upper(), logging.INFO),
0015         format="%(asctime)s %(levelname)s %(name)s %(message)s",
0016         handlers=handlers,
0017         force=True,
0018     )
0019
==== 文件开始 src/llmctg/models.py ====
0001 from __future__ import annotations
0002
0003 from dataclasses import asdict, dataclass, field
0004 from enum import Enum
0005 from typing import Any
0006
0007
0008 class ReadingLevel(str, Enum):
0009     PRIMARY = "primary"
0010     JUNIOR = "junior"
0011     SENIOR = "senior"
0012
0013     @property
```

源程序第 20 页 src/llmctg/models.py

```
0014     def chinese_name(self) -> str:
0015         return {self.PRIMARY: "小学", self.JUNIOR: "初中", self.SENIOR: "高中"}[self]
0016
0017     @property
0018     def numeric_label(self) -> int:
0019         return {self.PRIMARY: 0, self.JUNIOR: 1, self.SENIOR: 2}[self]
0020
0021     @classmethod
0022     def parse(cls, value: str | int | "ReadingLevel") -> "ReadingLevel":
0023         if isinstance(value, cls):
0024             return value
0025         aliases = {
0026             "0": cls.PRIMARY, "小学": cls.PRIMARY, "小学生": cls.PRIMARY,
0027             "primary": cls.PRIMARY,
0028             "1": cls.JUNIOR, "初中": cls.JUNIOR, "初中生": cls.JUNIOR,
0029             "junior": cls.JUNIOR,
0030             "2": cls.SENIOR, "高中": cls.SENIOR, "高中生": cls.SENIOR,
0031             "senior": cls.SENIOR,
0032         }
0033         key = str(value).strip().lower()
0034         if key not in aliases:
0035             raise ValueError(f"不支持的阅读等级: {value}")
0036         return aliases[key]
0037
0038
0039 @dataclass(slots=True)
0040 class GenerationRequest:
0041     topic: str
0042     level: ReadingLevel = ReadingLevel.JUNIOR
0043     instruction: str = ""
0044     keywords: list[str] = field(default_factory=list)
0045     count: int = 1
0046     seed: int | None = None
0047
0048     def validate(self) -> None:
0049         self.topic = self.topic.strip()
0050         if not self.topic:
0051             raise ValueError("主题不能为空")
0052         if not 1 <= self.count <= 20:
0053             raise ValueError("单次生成数量必须在 1 到 20 之间")
0054         self.level = ReadingLevel.parse(self.level)
0055         self.keywords = [str(item).strip() for item in self.keywords if str(item).strip()]
0056
0057
0058 @dataclass(slots=True)
0059 class TextStatistics:
0060     characters: int
0061     chinese_characters: int
0062     sentences: int
0063     paragraphs: int
```

```
0064     average_sentence_length: float
0065     max_sentence_length: int
0066     unique_character_ratio: float
0067     punctuation_ratio: float
0068
0069
0070 @dataclass(slots=True)
0071 class QualityMetrics:
0072     repetition_rate: float
0073     english_ratio: float
0074     uncommon_character_ratio: float
0075     structure_score: float
0076     length_score: float
0077     level_score: float
0078     overall_score: float
0079
0080
0081 @dataclass(slots=True)
0082 class EvaluationResult:
0083     target_level: ReadingLevel
0084     predicted_level: ReadingLevel
0085     quality_grade: str
0086     passed: bool
0087     statistics: TextStatistics
0088     metrics: QualityMetrics
0089     warnings: list[str] = field(default_factory=list)
0090     suggestions: list[str] = field(default_factory=list)
0091
0092     def to_dict(self) -> dict[str, Any]:
0093         data = asdict(self)
0094         data["target_level"] = self.target_level.value
0095         data["predicted_level"] = self.predicted_level.value
0096         return data
0097
0098
0099 @dataclass(slots=True)
0100 class GeneratedArticle:
0101     request_id: str
0102     topic: str
0103     level: ReadingLevel
0104     title: str
0105     content: str
0106     backend: str
0107     created_at: str
0108     evaluation: EvaluationResult | None = None
0109
0110     def to_dict(self) -> dict[str, Any]:
0111         data = asdict(self)
0112         data["level"] = self.level.value
0113         if self.evaluation:
```

源程序第 22 页 src/llmctg/models.py

```
0114         data["evaluation"] = self.evaluation.to_dict()
0115     return data
0116
===== 文件开始 src/llmctg/pipeline.py =====
0001 from __future__ import annotations
0002
0003 import logging
0004 from datetime import datetime, timezone
0005 from pathlib import Path
0006 from typing import Any, Iterable
0007
0008 from .config import AppConfig
0009 from .evaluator import ReadabilityEvaluator
0010 from .generators import TextGenerator, create_generator
0011 from .io_utils import append_jsonl, existing_ids, read_jsonl, write_json
0012 from .models import GeneratedArticle, GenerationRequest, ReadingLevel
0013 from .reporting import BatchSummary, build_summary, write_html_report, write_summary_csv
0014 from .text_utils import stable_id
0015
0016 LOGGER = logging.getLogger(__name__)
0017
0018
0019 class LLMCTGApplication:
0020     def __init__(self, config: AppConfig | None = None, generator: TextGenerator | None = None):
0021         self.config = config or AppConfig()
0022         self.generator = generator or create_generator(self.config.generation)
0023         self.evaluator = ReadabilityEvaluator(self.config.evaluation)
0024
0025     def generate(self, request: GenerationRequest) -> list[GeneratedArticle]:
0026         request.validate()
0027         raw_results = self.generator.generate(request)
0028         created_at = datetime.now(timezone.utc).astimezone().isoformat(timespec="seconds")
0029         articles: list[GeneratedArticle] = []
0030         for index, (title, content) in enumerate(raw_results):
0031             identifier = stable_id(request.topic, request.level.value, request.seed, index, content)
0032             evaluation = self.evaluator.evaluate(content, request.level)
0033             articles.append(
0034                 GeneratedArticle(
0035                     request_id=identifier,
0036                     topic=request.topic,
0037                     level=request.level,
0038                     title=title,
0039                     content=content,
0040                     backend=self.generator.name,
0041                     created_at=created_at,
0042                     evaluation=evaluation,
0043                 )
0044             )
0045         return articles
0046
```

源程序第 23 页 src/llmctg/pipeline.py

```
0047     def evaluate(self, text: str, level: ReadingLevel | str | int):
0048         return self.evaluator.evaluate(text, level)
0049
0050     def batch(self, input_path: str | Path, output_dir: str | Path, resume: bool = True) -> BatchSummary:
0051         output = Path(output_dir)
0052         output.mkdir(parents=True, exist_ok=True)
0053         records_path = output / "articles.jsonl"
0054         errors_path = output / "errors.jsonl"
0055         done = existing_ids(records_path) if resume else set()
0056         articles: list[GeneratedArticle] = []
0057         skipped = 0
0058         failed = 0
0059         for record in read_jsonl(input_path):
0060             source_id = stable_id(record.get("topic", ""), record.get("level", "junior"), record.get("_line_number"))
0061             if source_id in done:
0062                 skipped += 1
0063                 continue
0064             try:
0065                 request = self._request_from_record(record)
0066                 generated = self.generate(request)
0067                 for article in generated:
0068                     payload = article.to_dict()
0069                     payload["source_id"] = source_id
0070                     append_jsonl(records_path, payload)
0071                     articles.append(article)
0072                 done.add(source_id)
0073             except Exception as error: # 单条失败不终止批处理
0074                 failed += 1
0075                 LOGGER.exception("第 %s 行处理失败", record.get("_line_number"))
0076                 append_jsonl(errors_path, {
0077                     "source_id": source_id,
0078                     "line_number": record.get("_line_number"),
0079                     "error_type": type(error).__name__,
0080                     "message": str(error),
0081                     "record": {key: value for key, value in record.items() if not key.startswith("_")},
0082                 })
0083             summary = build_summary(articles, skipped=skipped, failed=failed)
0084             write_json(output / "summary.json", summary.to_dict())
0085             write_summary_csv(output / "summary.csv", articles)
0086             write_html_report(output / "report.html", articles, summary)
0087             return summary
0088
0089     @staticmethod
0090     def _request_from_record(record: dict[str, Any]) -> GenerationRequest:
0091         keywords = record.get("keywords", [])
0092         if isinstance(keywords, str):
0093             keywords = [item.strip() for item in keywords.replace(" ", "").split(",") if item.strip()]
0094         return GenerationRequest(
0095             topic=str(record.get("topic", "")),
0096             level=ReadingLevel.parse(record.get("level", "junior")),
```

源程序第 24 页 src/llmctg/pipeline.py

```
0097         instruction=str(record.get("instruction", "")),
0098         keywords=list(keywords),
0099         count=int(record.get("count", 1)),
0100         seed=int(record["seed"]) if record.get("seed") is not None else None,
0101     )
0102
0103     def close(self) -> None:
0104         self.generator.close()
0105
0106     def __enter__(self) -> "LLMCTGApplication":
0107         return self
0108
0109     def __exit__(self, exc_type: Any, exc: Any, traceback: Any) -> None:
0110         self.close()
0111
0112 ===== 文件开始 src/llmctg/reporting.py =====
0001 from __future__ import annotations
0002
0003 import html
0004 from dataclasses import asdict, dataclass
0005 from pathlib import Path
0006 from statistics import mean
0007
0008 from .io_utils import write_csv
0009 from .models import GeneratedArticle, ReadingLevel
0010 from .text_utils import write_text
0011
0012 @dataclass(slots=True)
0013 class BatchSummary:
0014     generated: int
0015     passed: int
0016     failed_quality: int
0017     skipped: int
0018     processing_errors: int
0019     average_score: float
0020     primary_count: int
0021     junior_count: int
0022     senior_count: int
0023
0024     def to_dict(self) -> dict[str, int | float]:
0025         return asdict(self)
0026
0027
0028
0029 def build_summary(
0030     articles: list[GeneratedArticle], skipped: int = 0, failed: int = 0
0031 ) -> BatchSummary:
0032     scores = [article.evaluation.metrics.overall_score for article in articles if article.evaluation]
0033     passed = sum(1 for article in articles if article.evaluation and article.evaluation.passed)
0034     return BatchSummary(
```

```

0035         generated=len(articles),
0036         passed=passed,
0037         failed_quality=len(articles) - passed,
0038         skipped=skipped,
0039         processing_errors=failed,
0040         average_score=round(mean(scores), 2) if scores else 0.0,
0041         primary_count=sum(article.level == ReadingLevel.PRIMARY for article in articles),
0042         junior_count=sum(article.level == ReadingLevel.JUNIOR for article in articles),
0043         senior_count=sum(article.level == ReadingLevel.SENIOR for article in articles),
0044     )
0045
0046
0047 def write_summary_csv(path: str | Path, articles: list[GeneratedArticle]) -> Path:
0048     rows = []
0049     for article in articles:
0050         evaluation = article.evaluation
0051         rows.append({
0052             "request_id": article.request_id,
0053             "topic": article.topic,
0054             "level": article.level.value,
0055             "title": article.title,
0056             "backend": article.backend,
0057             "quality_grade": evaluation.quality_grade if evaluation else "",
0058             "passed": evaluation.passed if evaluation else False,
0059             "overall_score": evaluation.metrics.overall_score if evaluation else "",
0060             "characters": evaluation.statistics.chinese_characters if evaluation else "",
0061             "predicted_level": evaluation.predicted_level.value if evaluation else "",
0062         })
0063     return write_csv(path, rows, [
0064         "request_id", "topic", "level", "title", "backend", "quality_grade",
0065         "passed", "overall_score", "characters", "predicted_level",
0066     ])
0067
0068
0069 def write_html_report(
0070     path: str | Path, articles: list[GeneratedArticle], summary: BatchSummary
0071 ) -> Path:
0072     cards = "\n".join(_article_card(article) for article in articles)
0073     document = f"""<!doctype html>
0074 <html lang="zh-CN">
0075 <head>
0076 <meta charset="utf-8">
0077 <meta name="viewport" content="width=device-width,initial-scale=1">
0078 <title>中文分级阅读内容质量报告</title>
0079 <style>
0080 body{{font-family:"Microsoft YaHei",sans-serif;margin:0;background:#f4f6f8;color:#1f2937}}
0081 main{{max-width:1080px;margin:0 auto;padding:32px 20px}}
0082 h1{{font-size:28px;margin:0 0 8px}} .sub{{color:#64748b;margin-bottom:24px}}
0083 .summary{{display:grid;grid-template-columns:repeat(auto-fit,minmax(150px,1fr));gap:12px;margin:20px 0 28px}}
0084 .metric,.card{{background:#fff;border:1px solid #e2e8f0;border-radius:10px;padding:16px}}

```

源程序第 26 页 src/llmctg/reporting.py

```
0085 .metric strong{{display:block;font-size:25px;color:#0f4c81}} .card{{margin:14px 0}}
0086 .meta{{color:#64748b;font-size:14px}} .content{{white-space:pre-wrap;line-height:1.85}}
0087 .pass{{color:#177245}} .fail{{color:#b42318}} ul{{line-height:1.7}}
0088 </style>
0089 </head>
0090 <body><main>
0091 <h1>中文分级阅读内容质量报告</h1><div class="sub">批量生成与自动评估结果</div>
0092 <section class="summary">
0093 <div class="metric"><strong>{summary.generated}</strong>生成数量</div>
0094 <div class="metric"><strong>{summary.passed}</strong>质量通过</div>
0095 <div class="metric"><strong>{summary.average_score}</strong>平均得分</div>
0096 <div class="metric"><strong>{summary.processing_errors}</strong>处理异常</div>
0097 </section>{cards or '本次没有新增文章。<p>'}
0098 </main></body></html>""
0099     return write_text(path, document)
0100
0101
0102 def _article_card(article: GeneratedArticle) -> str:
0103     evaluation = article.evaluation
0104     if evaluation is None:
0105         return ""
0106     status_class = "pass" if evaluation.passed else "fail"
0107     status = "通过" if evaluation.passed else "待改进"
0108     warnings = "".join(f"<li>{html.escape(item)}</li>" for item in evaluation.warnings)
0109     return f""<article class="card">
0110 <h2>{html.escape(article.title)}</h2>
0111 <div class="meta">{html.escape(article.level.chinese_name)} · {html.escape(article.backend)} ·
0112 <span class="{status_class}">{status} {evaluation.metrics.overall_score} 分</span></div>
0113 <p class="content">{html.escape(article.content)}</p>
0114 <p><b>预测等级:</b> {html.escape(evaluation.predicted_level.chinese_name)};
0115 <b>汉字数:</b> {evaluation.statistics.chinese_characters};
0116 <b>平均句长:</b> {evaluation.statistics.average_sentence_length:.1f}</p>
0117 {('<p><b>检查提示</b></p><ul>' + warnings + '</ul>') if warnings else ''}
0118 </article>""
0119
0120
0121 ===== 文件开始 src/llmctg/text_utils.py =====
0122 from __future__ import annotations
0123
0124
0125 import hashlib
0126 import re
0127 import unicodedata
0128 from collections import Counter
0129 from pathlib import Path
0130
0131
0132 CHINESE_PATTERN = re.compile(r"[\u3400-\u4dbf\u4e00-\u9fff]")
0133 SENTENCE_SPLIT_PATTERN = re.compile(r"(?<=[。! ? !? ;])")
0134 WHITESPACE_PATTERN = re.compile(r"[ \t\r\n\f\v]+")
0135
0136
0137 def normalize_text(text: str) -> str:
```

源程序第 27 页 src/llmctg/text_utils.py

```
0015     text = unicodedata.normalize("NFKC", str(text or ""))
0016     text = text.replace("\r\n", "\n").replace("\r", "\n")
0017     lines = [WHITESPACE_PATTERN.sub(" ", line).strip() for line in text.split("\n")]
0018     return "\n".join(line for line in lines if line).strip()
0019
0020
0021 def ensure_terminal_punctuation(text: str) -> str:
0022     text = normalize_text(text)
0023     if text and text[-1] not in "。! ? !?...\"'\"' ] ) )":
0024         return text + "。"
0025     return text
0026
0027
0028 def split_sentences(text: str) -> list[str]:
0029     normalized = normalize_text(text)
0030     if not normalized:
0031         return []
0032     parts = SENTENCE_SPLIT_PATTERN.split(normalized.replace("\n", ""))
0033     return [item.strip() for item in parts if item.strip()]
0034
0035
0036 def split_paragraphs(text: str) -> list[str]:
0037     return [line.strip() for line in normalize_text(text).split("\n") if line.strip()]
0038
0039
0040 def chinese_characters(text: str) -> list[str]:
0041     return CHINESE_PATTERN.findall(text)
0042
0043
0044 def character_ngrams(text: str, size: int = 4) -> list[str]:
0045     chars = chinese_characters(text)
0046     if size <= 0:
0047         raise ValueError("n-gram 长度必须为正整数")
0048     return ["".join(chars[index:index + size]) for index in range(max(0, len(chars) - size + 1))]
0049
0050
0051 def repetition_rate(text: str, size: int = 4) -> float:
0052     grams = character_ngrams(text, size=size)
0053     if not grams:
0054         return 0.0
0055     counts = Counter(grams)
0056     repeated = sum(count - 1 for count in counts.values() if count > 1)
0057     return repeated / len(grams)
0058
0059
0060 def stable_id(*values: object, length: int = 16) -> str:
0061     joined = "\x1f".join(str(value) for value in values)
0062     return hashlib.sha256(joined.encode("utf-8")).hexdigest()[length:]
0063
0064
```

源程序第 28 页 src/llmctg/text_utils.py

```
0065 def safe_filename(value: str, fallback: str = "output") -> str:
0066     cleaned = re.sub(r"[<>:\\"/\\|]?*\x00-\x1f]", "_", normalize_text(value))
0067     cleaned = cleaned.strip("._")
0068     return (cleaned or fallback)[:80]
0069
0070
0071 def read_text(path: str | Path) -> str:
0072     with Path(path).open("r", encoding="utf-8-sig") as stream:
0073         return stream.read()
0074
0075
0076 def write_text(path: str | Path, content: str) -> Path:
0077     target = Path(path)
0078     target.parent.mkdir(parents=True, exist_ok=True)
0079     with target.open("w", encoding="utf-8", newline="\n") as stream:
0080         stream.write(content)
0081     return target
0082
0083
0084 ===== 文件开始 src/llmctg/training.py =====
0085
0086 from __future__ import annotations
0087
0088
0089 from dataclasses import dataclass
0090 from pathlib import Path
0091 from typing import Any
0092
0093
0094 @dataclass(slots=True)
0095 class FineTuneOptions:
0096     model_path: str
0097     dataset_path: str
0098     output_dir: str
0099     epochs: float = 3.0
0100     batch_size: int = 1
0101     gradient_accumulation_steps: int = 4
0102     learning_rate: float = 2e-4
0103     max_length: int = 1024
0104     lora_rank: int = 8
0105     lora_alpha: int = 32
0106     lora_dropout: float = 0.05
0107     device: str = "auto"
0108
0109     def validate(self) -> None:
0110         if not Path(self.model_path).exists():
0111             raise FileNotFoundError(f"基础模型目录不存在: {self.model_path}")
0112         if not Path(self.dataset_path).exists():
0113             raise FileNotFoundError(f"训练数据不存在: {self.dataset_path}")
0114         if self.epochs <= 0 or self.batch_size <= 0 or self.max_length < 64:
0115             raise ValueError("训练轮数、批量大小和最大长度必须为有效正数")
0116
0117
0118
0119
0120
0121
0122
0123
0124
0125
0126
0127
0128
0129
0130
0131
```

源程序第 29 页 src/llmctg/training.py

```
0032 class LoraTrainer:
0033     """把原始微调脚本封装为显式配置、可验证的训练服务。"""
0034
0035     def __init__(self, options: FineTuneOptions):
0036         options.validate()
0037         self.options = options
0038
0039     def train(self) -> dict[str, Any]:
0040         try:
0041             import torch
0042             from datasets import load_dataset
0043             from peft import LoraConfig, TaskType, get_peft_model
0044             from transformers import (
0045                 AutoModelForCausalLM,
0046                 AutoTokenizer,
0047                 DataCollatorForLanguageModeling,
0048                 Trainer,
0049                 TrainingArguments,
0050             )
0051         except ImportError as error:
0052             raise RuntimeError("缺少训练依赖, 请安装 torch、transformers、datasets 和 peft") from error
0053
0054         device = self.options.device
0055         if device == "auto":
0056             device = "cuda" if torch.cuda.is_available() else "cpu"
0057         dtype = torch.float16 if device.startswith("cuda") else torch.float32
0058         tokenizer = AutoTokenizer.from_pretrained(
0059             self.options.model_path, trust_remote_code=False, use_fast=True
0060         )
0061         if tokenizer.pad_token_id is None:
0062             tokenizer.pad_token = tokenizer.eos_token
0063         base_model = AutoModelForCausalLM.from_pretrained(
0064             self.options.model_path, trust_remote_code=False, torch_dtype=dtype
0065         )
0066         lora = LoraConfig(
0067             task_type=TaskType.CAUSAL_LM,
0068             inference_mode=False,
0069             r=self.options.lora_rank,
0070             lora_alpha=self.options.lora_alpha,
0071             lora_dropout=self.options.lora_dropout,
0072             target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
0073         )
0074         model = get_peft_model(base_model, lora)
0075         model.config.use_cache = False
0076         if device.startswith("cuda"):
0077             model.gradient_checkpointing_enable()
0078             model.enable_input_require_grads()
0079         model.to(device)
0080
0081         dataset = load_dataset("json", data_files={"train": self.options.dataset_path})["train"]
```

```

0082
0083     def tokenize(record: dict[str, Any]) -> dict[str, Any]:
0084         instruction = str(record.get("instruction", "")).strip()
0085         input_text = str(record.get("input", "")).strip()
0086         output_text = str(record.get("output", "")).strip()
0087         prompt = instruction + ("\n" + input_text if input_text else "")
0088         full_text = prompt + "\n" + output_text
0089         encoded = tokenizer(
0090             full_text,
0091             truncation=True,
0092             max_length=self.options.max_length,
0093             padding=False,
0094         )
0095         encoded["labels"] = encoded["input_ids"].copy()
0096         return encoded
0097
0098     tokenized = dataset.map(tokenize, remove_columns=dataset.column_names)
0099     arguments = TrainingArguments(
0100         output_dir=self.options.output_dir,
0101         overwrite_output_dir=True,
0102         num_train_epochs=self.options.epochs,
0103         per_device_train_batch_size=self.options.batch_size,
0104         gradient_accumulation_steps=self.options.gradient_accumulation_steps,
0105         learning_rate=self.options.learning_rate,
0106         fp16=device.startswith("cuda"),
0107         save_strategy="epoch",
0108         save_total_limit=2,
0109         logging_steps=10,
0110         report_to="none",
0111         dataloader_num_workers=0,
0112         remove_unused_columns=False,
0113     )
0114     trainer = Trainer(
0115         model=model,
0116         args=arguments,
0117         train_dataset=tokenized,
0118         data_collator=DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False),
0119     )
0120     result = trainer.train()
0121     model.save_pretrained(self.options.output_dir)
0122     tokenizer.save_pretrained(self.options.output_dir)
0123     return {
0124         "output_dir": str(Path(self.options.output_dir).resolve()),
0125         "train_samples": len(tokenized),
0126         "train_loss": float(result.training_loss),
0127         "global_step": int(result.global_step),
0128         "device": device,
0129     }
0130

```